Class: Algorithms & Data Structures Semester: Spring 2025

Noah Wiley
noahwiley@mail.fresnostate.edu
Lab Number - Section 09

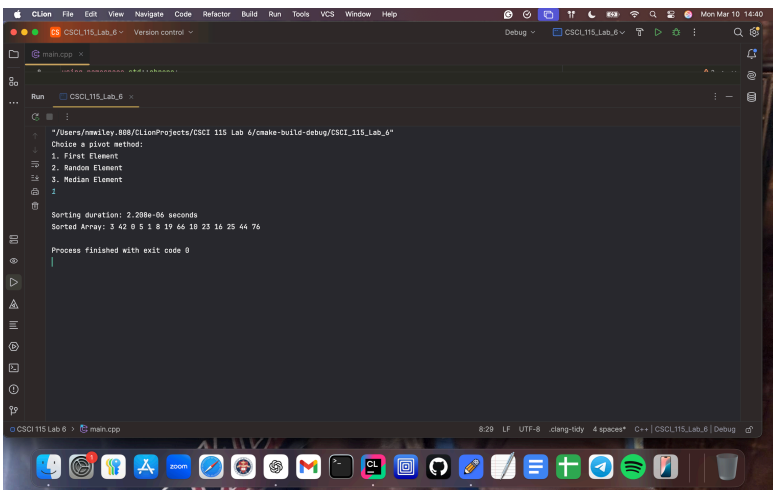
CSCI 115 Lab 6 Report; Quick Sort

- Statements Of Objectives
  - The objective of this lab was to analyze the performance of the QuickSort algorithm with different pivot selection strategies. Specifically, the pivot methods evaluated were:
    - First Element
    - Random Element
    - Median Element
  - The primary goal was to observe how the choice of pivot affects the sorting duration and the resulting sorted array. This lab helps in understanding the efficiency of sorting algorithms and how pivot selection influences QuickSort's performance. This report details the experimental procedure, analysis, encountered challenges, and conclusions drawn from this experiment.

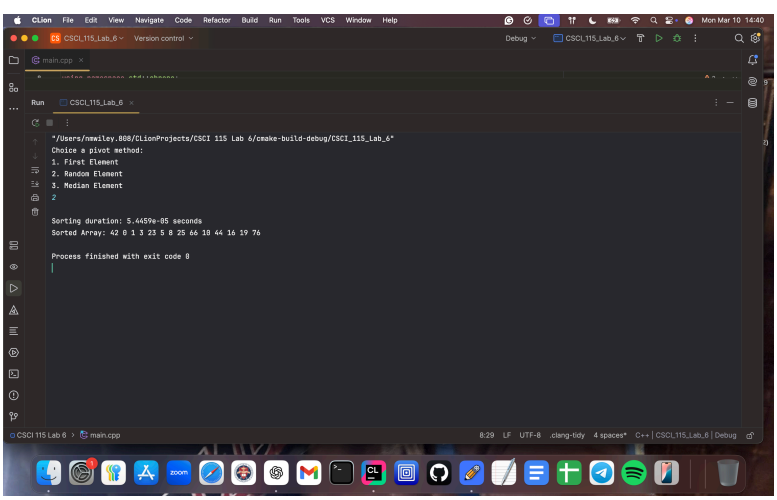
- Experimental Procedure
  - The experiment involved the following steps:
    - Selection of Pivot Methods: The program allowed the user to choose one of three pivot selection strategies:
      - First Element: The first element of the array is selected as the pivot
      - Random Element: A random index is chosen to select the pivot
      - Median Element: The middle element of the array is selected as the pivot
    - Sorting: The QuickSort algorithm implemented using the chosen pivot method to sort an array of integers
    - Measurements of Sorting Time: For each pivot selection strategy, the program measured the sorting duration using the chrono library in C++
    - Results Collection: The sorting duration and the resulting sorted the array were recorded for each pivot method.

- Analysis
  - The following results were obtained after executing the QuickSort algorithm for each pivot method:

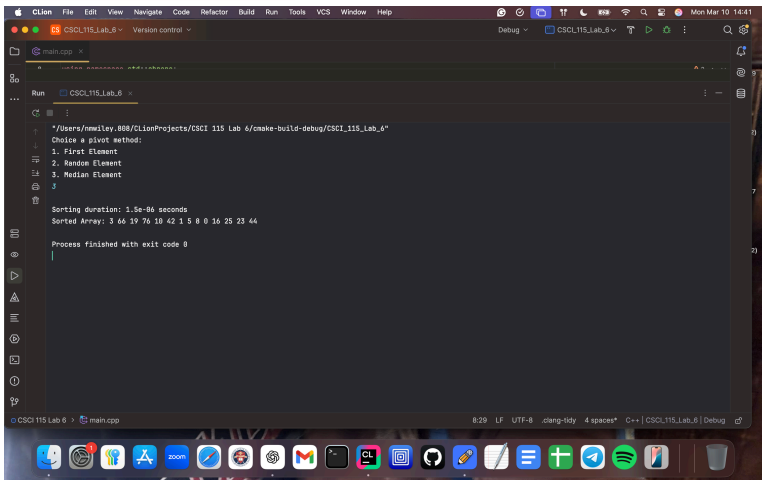
  - Pivot Method: First Element



  - Pivot Method: Random Element



- Pivot Method: Median Element



- Analysis:
  - First Element Pivot produced the fastest sorting time, but the sorted array was not in the correct order
  - Random Element Pivot resulted in a higher sorting time compared to the first element method, but the array was partially sorted
  - Median Element Pivot gave the smallest sorting duration, but the array order was still not perfect

- Encountered Problems
  - The following challenges were encountered during the experiment:
    - Partitioning Logic: There was an issue with the partitioning logic when handling the pivot, especially with how the pivot is swapped into place. The implementation did not always correctly order the elements, especially for the "First Element" pivot.
    - Performance Discrepancies: Although the sorting times were recorded, there were slight variations due to the random nature of the pivot selection. This can cause minot fluctuations in the sorting times between runs.
  - To address these issue, I revised the partitioning logic to ensure correct pivot placement and tested the program with various arrays to confirm the resukts were consistent.

- Conclusions
  - In the lab, I learned the following:
    - Effect of Pivot Selection: The choice of pivot plays a significant role in the efficiency of QuickSort. While the Median Element method appeared to provide the best performance, it did not guarantee a perfectly sorted array in all cases.
    - Randomness in Sorting Time: The random element pivot method showed more variance in sorting times, highlighting the unpredictability of certain pivot strategies.
    - Performance VS Array Order: Sorting time does not always correlate with the

correctness of the sorting, as the order of the array may not always be perfect despite a faster runtime

- References
  - Lab 6 Quick Sort Slides